ST. JOSEPH'S COLLEGE
OF ENGINEERING

CYBER SECURITY CHATBOT ASSISTANT

BACHELOR OF ENGINEERING (B.E)

IN

CYBER SECURITY

A PROJECT BY

K Kevin Paul

&

P S Ahmad Anas

ABSTRACT

This project introduces a simple AI-powered chatbot designed to assist users with cybersecurity-related questions. Built using Python and natural language processing (NLP) techniques, the chatbot serves as a quick and accessible tool for students, professionals, and the public to learn about cybersecurity concepts, best practices, and threat prevention. It leverages a predefined knowledge base and machine learning models to provide accurate, user-friendly responses to queries on topics such as phishing, malware, password safety, network attacks, and more. The chatbot aims to promote cybersecurity awareness and literacy through conversational interaction.

TABLE OF CONTENTS

OBJECTIVES:

1.INTRODUCTION [PAGE – 4]

2.OBJECTIVE AND FUNCTIONALITY[PAGE – 5]

3.DESIGN[PAGE – 6]

4.IMPLEMENTATION[PAGE – 7,8]

5.EXPLANATION[PAGE – 8,9,10]

6.CODE[PAGE – 11,12]

7.OUTPUT[PAGE – 13,14]

8.CONCLUSION[PAGE – 15,16,17]

INTRODUCTION

Cybersecurity has become a critical aspect of modern technology usage. This project focuses on building a chatbot capable of providing instant answers to cybersecurity-related questions. Unlike generic assistants, this bot is trained or programmed with cybersecurity-specific topics to help users learn about threats and defenses effectively. It demonstrates the practical application of AI and natural language processing in cybersecurity education and assistance.

OBJECTIVES AND FUNCTIONALITIES

Main Objective:

To create a chatbot that can respond to user queries related to cybersecurity topics using an AI-powered question-answering system.

Key Functionalities:

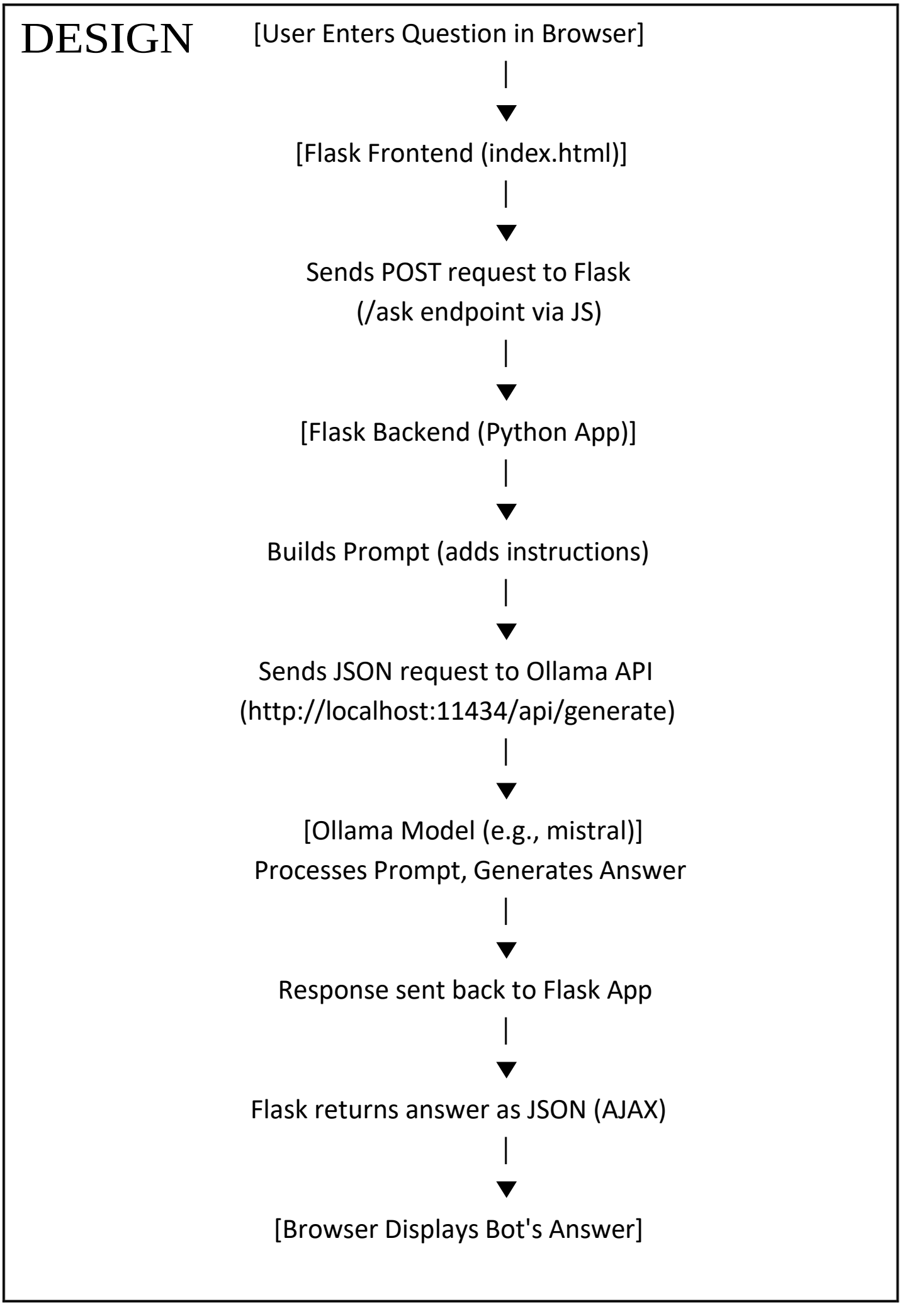
1. Query Handling: Accepts text input from users.

2. Knowledge Retrieval: Answers questions using a predefined knowledge base or trained model.

3. Topic Coverage: Covers key cybersecurity areas – malware, phishing, firewalls, encryption, social engineering, etc.

4. User Interaction: Provides a simple conversational interface for ease of use.

DESIGN


[User Enters Question in Browser]
[Flask Frontend (index.html)]
Sends POST request to Flask
(/ask endpoint via JS)
[Flask Backend (Python App)]
Builds Prompt (adds instructions)
Sends JSON request to Ollama API
(http://localhost:11434/api/generate)
[Ollama Model (e.g., mistral)]
Processes Prompt, Generates Answer
Response sent back to Flask App
Flask returns answer as JSON (AJAX)
[Browser Displays Bot's Answer]

IMPLEMENTATION

```
from flask import Flask, render_template, request, jsonify
import requests

app = Flask(__name__)

@app.route("/")
def home():
return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
user_question = request.json.get("question", "")

prompt = f"You are a cybersecurity expert. Provide clear, concise answers.\n\nQuestion: {user_question}"


 try:
      # Set num_predict lower to make it faster
      response = requests.post("http://localhost:11434/api/generate", json={
         "model": "mistral",          # Use fastest model
         "prompt": prompt,
         "num_predict": 50,           # Reduce number of tokens
         "stream": False              # No streaming needed
      })

      if response.status_code == 200:
         answer = response.json().get("response", "").strip()
         return jsonify({"answer": answer})

  else:
     return jsonify({"answer": f"Error from Ollama: {response.status_code} - {response.text}"})
```

```
except Exception as e:
    return jsonify({"answer": f"Error: {str(e)}"})

if __name__ == "__main__":
    app.run(debug=True)
```

EXPLANATION

1. User Enters a Question in the Chat Interface

The user types a question related to cybersecurity (e.g., "How do I prevent phishing attacks?") in a text box on the webpage and submits it.

This input is handled by JavaScript on the frontend, which sends the question to the Flask backend using a **POST request** to the /ask route.

2. Flask Receives the Question

The Flask server listens for incoming requests on the /ask route.

It retrieves the question from the JSON body of the request using request.json.get("question", "").

3. Prompt is Prepared

Flask adds a predefined instruction to the question:

"You are a cybersecurity expert. Provide clear, concise answers."

This combined text becomes the **prompt** that will be sent to the AI model.

Example prompt:

You are a cybersecurity expert. Provide clear, concise answers.\n\nQuestion: How do I prevent phishing attacks?

4. Flask Sends the Prompt to Ollama

Flask makes a **POST request** to the local Ollama server (http://localhost:11434/api/generate).

It includes the prompt and specifies:

Which model to use (e.g., "mistral")

How many tokens to predict (e.g., 100)

Whether to stream the output (set to True)

5. Ollama Processes the Prompt

Ollama receives the prompt and sends it to the selected language model (like **Mistral**).

The model processes the prompt and generates a relevant response based on the question.

Example response:

“To prevent phishing attacks, never click on suspicious links, verify email sources, and enable multi-factor authentication.”

6. Flask Receives the Response from Ollama

The response is returned to Flask in JSON format.

Flask extracts the actual answer text from the "response" field.

If there's an error, Flask catches it and returns an error message.

7. Flask Sends the Answer Back to the Frontend

Flask sends a **JSON response** back to the user's browser.

It contains the chatbot's answer in this format:

{"answer": "To prevent phishing attacks, ..."}

8. Frontend Displays the Answer

The frontend (HTML + JavaScript) receives the JSON response.

It then dynamically updates the chat window by:

Creating a message bubble

Inserting the AI's answer inside it

Displaying it under the user's question

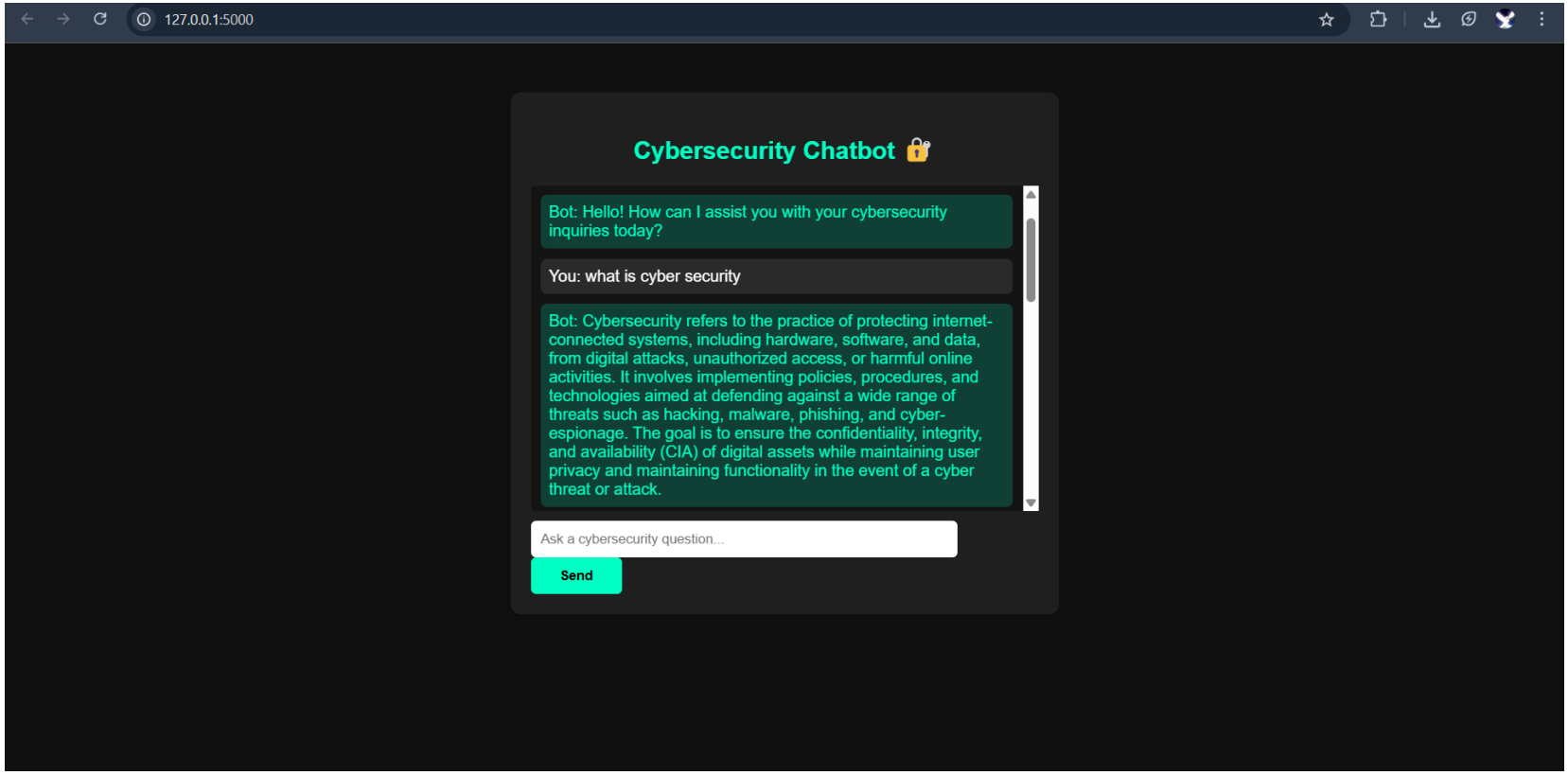
✅ **Final Result**

The user now sees the chatbot's answer on the webpage, just like a real-time conversation.

The entire process happens within a few seconds.

CODE

```python
from flask import Flask, render_template, request, jsonify
import requests

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    user_question = request.json.get("question", "")
    prompt = f"You are a cybersecurity expert. Provide clear, concise answers.\n\nQuestion: {user_question}"

    try:
        # Set num_predict lower to make it faster
        response = requests.post("http://localhost:11434/api/generate", json={
            "model": "mistral",           # Use fastest model
            "prompt": prompt,
            "num_predict": 50,            # Reduce number of tokens
            "stream": False               # No streaming needed
        })

        if response.status_code == 200:
            answer = response.json().get("response", "").strip()
            return jsonify({"answer": answer})
        else:
            return jsonify({"answer": f"Error from Ollama: {response.status_code} - {response.text}"})

    except Exception as e:
        return jsonify({"answer": f"Error: {str(e)}"})

if __name__ == "__main__":
    app.run(debug=True)
```


127.0.0.1:5000
Cybersecurity Chatbot
Bot: Hello! How can I assist you with your cybersecurity inquiries today?
You: what is cyber security
Bot: Cybersecurity refers to the practice of protecting internet-connected systems, including hardware, software, and data, from digital attacks, unauthorized access, or harmful online activities. It involves implementing policies, procedures, and technologies aimed at defending against a wide range of threats such as hacking, malware, phishing, and cyber-espionage. The goal is to ensure the confidentiality, integrity, and availability (CIA) of digital assets while maintaining user privacy and maintaining functionality in the event of a cyber threat or attack.
Ask a cybersecurity question...
Send

OUTPUT

1. User Question Displayed

When the user enters a question on the chatbot interface, it appears on the screen as their message in the conversation.

2. Bot Response Displayed

Below the user's message, the bot responds with a clear, informative answer related to the question asked.

3. Answer is Concise and Cybersecurity-Focused

The response is usually brief, easy to understand, and directly related to cybersecurity, because the AI is instructed to answer like a cybersecurity expert.

4. Real-Time Chat Appearance

The conversation is shown in a chat format, making it feel like the user is interacting with a live assistant.

5. Dynamic Interaction

The user can ask multiple questions, and each time the bot processes the new input and adds its answer to the conversation flow.

6. Model-Generated Content

The answers are generated by an AI model running locally, meaning the responses may vary slightly each time, depending on the phrasing and model used.

7. No Internet Required for AI Response

Because the AI model is hosted locally using tools like Ollama, the response is generated without depending on external servers or APIs.

8. Output Format is Plain Text

The answers are shown in plain text (unless customized further), without formatting like bold, bullet points, or images.

CONCLUSION

Conclusion: Effective Use Cases for the Cybersecurity Chatbot

This cybersecurity-focused chatbot built using **Python**, **Flask**, and **Ollama** has significant potential across various domains, especially in areas where **real-time, accurate cybersecurity advice** is critical. The following are some of the most effective use cases for this chatbot:

1. Cybersecurity Awareness and Training

- **Internal Use in Organizations**: This chatbot can be implemented within organizations to help employees understand cybersecurity best practices, potential threats, and how to handle sensitive information. It can provide on-the-spot guidance on topics like phishing, password security, and safe internet practices.
- **Training Platforms**: The chatbot can be used in training programs or cybersecurity education platforms to help new learners get real-time answers to their questions regarding network security, ethical hacking, and more.

2. Incident Response and Support

- **Automated First-Response Assistance**: In the event of a cybersecurity incident (like a data breach or malware attack), the chatbot can guide users through initial response steps. It can provide instructions for isolating compromised systems, reporting incidents, and conducting immediate triage.
- **Help Desk Automation**: For businesses with dedicated IT or cybersecurity teams, this chatbot can serve as the first point of contact, filtering and addressing common inquiries before escalating to human experts.

3. Cybersecurity Solutions for Small Businesses

Affordable and Scalable Security: Small and medium-sized businesses (SMBs) can use this chatbot to improve their security posture without the need for extensive cybersecurity teams. The chatbot can offer tailored advice on firewalls, encryption, secure software practices, and more, based on the specific needs of the business.

- **User-Friendly Guidance**: Non-technical users in small businesses can ask the chatbot questions about securing their networks and devices, ensuring they adhere to industry standards for cybersecurity.

4. 24/7 Security Monitoring Assistance

- **Real-Time Threat Intelligence**: The chatbot can be integrated with real-time threat intelligence systems to offer proactive security advice. For instance, if a new vulnerability or attack vector emerges, the chatbot could alert users and provide recommendations on mitigating risks.
- **Self-Help Support**: Users can interact with the chatbot at any time to get assistance on securing their devices or understanding alerts from their security monitoring systems.

5. Cybersecurity Compliance and Audit

Compliance Guidance: Organizations that need to adhere to regulations like **GDPR**, **HIPAA**, or **PCI-DSS** can use the chatbot to ensure they are following the right processes and protocols. It can offer real-time advice on how to comply with data protection and privacy laws.

Audit Assistance: For businesses undergoing cybersecurity audits, the chatbot can help prepare necessary documentation, answer audit-related questions, and provide assistance during the auditing process.

6. Public Information and Cybersecurity Awareness Campaigns

Public Awareness: The chatbot can be deployed on websites or apps of government organizations, educational institutions, or cybersecurity NGOs to educate the public about the latest cyber threats. It could help users understand basic cybersecurity concepts like encryption, secure browsing, and protecting personal data.

Awareness Campaigns: It can also be a tool for large-scale public cybersecurity awareness campaigns, where users can interact with the chatbot to get more information about how to protect themselves online.

7. Integration with Existing Security Platforms

Integration with Security Dashboards: For larger organizations using SIEM (Security Information and Event Management) systems or other cybersecurity tools, this chatbot can integrate with those platforms. It can interpret logs, respond to alerts, and assist security analysts by providing quick insights into potential threats.

Conclusion

In summary, this chatbot can be effectively deployed in **cybersecurity education**, **incident response**, **compliance management**, and **real-time assistance**. Its ability to provide immediate, accurate, and actionable cybersecurity advice makes it an invaluable tool in both **enterprise environments** and **public-facing applications**. Whether it's helping businesses enhance their security practices, aiding individuals in staying safe online, or assisting organizations in managing complex security incidents, this chatbot offers a scalable, cost-effective solution to improving cybersecurity awareness and response.

REFERENCE

https://github.com/KEVIN-20065473/CYBER-SECURITY---CHATBOT

https://chatgpt.com/c/68122032-8124-8011-9d8c-a33675bfa694

Thank You